

成熟版意图原生生成式 UI 通用架构设计与构建手册

中文增强阅读版

文件名: mature-intent-native-generative-ui-build-guide1.md

更新时间: 2026-07-05

阅读前说明

这份文档解释一种当前最现实、最可产品化的下一代软件架构:

用户用自然语言表达目标, 软件用受控的结构化计划、能力系统、权限策略和生成式 UI 运行时, 把目标转化为真实可执行、可审计、可恢复的界面和动作。

你可以把它理解为:

不是“AI 聊天框”

也不是“让模型乱写前端代码”

而是“一个由自然语言驱动的软件操作系统”

本文尽量使用中文表达。少量英文术语会保留在括号中, 因为这些词在论文、开源框架和官方文档中经常直接出现。

| 一页读懂

成熟方案的核心判断：

用户表达可以自由。

模型输出必须结构化。

界面生成必须声明式。

组件必须来自注册表。

动作必须绑定能力。

权限必须由策略引擎裁决。

执行必须进入沙箱。

体验必须经过校验。

结果必须可以审计和恢复。

最重要的架构选择：

问题	成熟答案
模型能不能直接写 UI 代码？	不能。只能输出受控 UI DSL。
模型能不能直接调用数据库？	不能。只能引用注册过的数据源。
动态按钮能不能直接执行函数？	不能。只能触发 ActionRef，再进入策略引擎。
用户自然语言能不能控制一	可以控制 “已注册且有权限的能力” 。

问题	成熟答案
切?	
UI 能不能自由变化?	可以, 但必须经过组件注册、布局求解和 UX 校验。
最适合 MVP 的路线是什么?	受控声明式生成 UI, 而不是开放式代码生成。

推荐阅读路径

如果你只想先建立概念:

1. 读第 0-5 节。
2. 看 [BRAINSTORM/mature-intent-native-generative-ui-build-guide.html](#)。
3. 再读第 21-32 节的构建步骤。

如果你准备动手实现:

1. 先做第 21 节“四张清单”。
2. 再做第 22-25 节: 组件库、UI DSL、动态画布、动作路由。
3. 然后做第 26-28 节: 规则规划器、模型网关、策略引擎。
4. 最后做第 29-33 节: UX 校验、流式 UI、测试矩阵、生产化。

中文术语表

英文术语	本文中文名	简短解释
Intent Runtime	意图运行时	理解用户目标、生成计划、执行能力的核心运行

英文术语	本文中文名	简短解释
		时。
Generative UI Runtime	生成式 UI 运行时	把 UI DSL 渲染成动态界面的运行时。
UI DSL	UI 领域描述语言	一种 JSON 风格的界面配置，不是前端代码。
Capability Graph	能力图	软件所有可执行能力的注册表和依赖图。
Policy Engine	策略引擎	独立裁决权限、风险、确认和拒绝的系统。
Dynamic Canvas	动态画布	接收 UI DSL 并渲染真实界面的容器。
Component Card	组件卡片	告诉模型“有哪些组件、能传什么属性”的说明卡。
Design Tokens	设计令牌	颜色、字号、间距、密度等设计系统变量。
ActionRef	动作引用	动态 UI 中按钮触发的受控动作描述。
UX Validator	用户体验校验器	检查遮挡、坏弹窗、可达性和误导性 UI。
Execution Journal	执行日志	记录输入、计划、策略、执行和状态变化。

十步构建总览

第 0 步：列注册表

组件、能力、数据源、风险规则。

第 1 步：定义协议

IntentPlan、UI DSL、ActionRef、DataSourceRef。

第 2 步：实现动态画布

手写 DSL 可以渲染，错误 DSL 会拒绝。

第 3 步：实现动作路由

所有动态按钮都进入策略引擎。

第 4 步：实现规则规划器

不接模型也能跑完整链路。

第 5 步：接入大模型网关

OpenAI、DeepSeek、本地模型都只是 provider。

第 6 步：实现策略引擎

高风险动作确认，越权动作拒绝。

第 7 步：实现 UX 校验

检查遮挡、坏弹窗、不可达按钮和误导文案。

第 8 步：实现流式 UI

先显示计划，再显示骨架，最后逐步挂载组件

第 9 步：建立测试矩阵

意图样例、Schema、策略、视觉回归和红队攻击。

0. 结论先行

目前最成熟、最现实、最可产品化的范式，不是“让大模型自由生成 UI 代码”，而是：

Controlled Declarative Generative UI + Agentic Intent Runtime

中文可以称为：

受控声明式生成 UI + 意图智能体运行时。

它的核心结构是：

用户自然语言

-> 意图运行时生成结构化计划

-> 策略引擎裁决权限和风险

-> 能力执行器改变真实状态

-> 生成式 UI 运行时输出受控 UI DSL

-> 动态画布渲染组件树

-> UX Validator 检查体验风险

-> 审计系统记录全过程

一句话：

模型负责理解和建议，系统负责验证、裁决、执行和兜底。

1. 这份文档如何使用

如果你是新手，推荐按这个顺序读：

1. 读第 2-5 节，先建立正确心智模型。
2. 读第 6-9 节，理解完整架构和模块分工。
3. 读第 10-16 节，理解核心协议：IntentPlan、UI DSL、Capability、Policy。
4. 读第 17-25 节，按步骤构建一个 MVP。
5. 读第 26-32 节，进入生产级安全、体验、测试和演进。

配套可视化：

- [BRAINSTORM/mature-intent-native-generative-ui-build-guide.mmd](#)
- [BRAINSTORM/mature-intent-native-generative-ui-build-guide.html](#)

参考资料：

- BRAINSTORM/reference/reading-notes.md
- BRAINSTORM/reference/reasoning-for-mobile-user-experience-with-multimodal-llms-2606.13192.pdf
- BRAINSTORM/L-GUI Architecture_ The Generative UI Paradigm.pdf

2. 范式边界

这个范式解决的问题：

- 用户不想学习复杂软件路径，只想表达目标。
- 软件 UI 不应固定死，而应根据任务动态重组。
- 复杂目标应由系统拆解成多步工作流。
- 所有动作仍必须有权限、安全、审计和恢复。

这个范式不做的事：

- 不让 LLM 直接写生产 UI 代码。
- 不让 LLM 直接操作 DOM。
- 不让 LLM 直接写数据库。
- 不让动态生成按钮绕过权限。
- 不用 prompt 代替安全系统。

成熟系统的判断标准：

Natural language is flexible.

Plan is structured.

UI is declarative.

Components are registered.

Actions are capability-bound.

Policy is authoritative.

Execution is sandboxed.

UX is validated.

Audit is mandatory.

Fallback is always available.

3. 为什么“受控声明式”最成熟

生成式 UI 有三条路线。

3.1 Controlled Generative UI

开发者完全控制组件和布局，模型只选择少量 preset。

优点：

- 最安全。
- 最容易测试。
- 体验最稳定。

缺点：

- 自由度有限。

适合：

- MVP。

- 金融、医疗、企业管理、开发工具等高可信产品。

3.2 Declarative Generative UI

模型输出 JSON DSL，系统把 DSL 映射成组件树。

优点：

- 自由度高。
- 仍可验证、可审计、可回滚。
- 是目前最适合产品化的平衡点。

缺点：

- 需要设计 DSL、组件库、布局求解和校验系统。

适合：

- 你的目标范式。
- 大多数严肃产品的未来形态。

3.3 Open-ended Generative UI

模型直接生成 HTML/CSS/JS 或直接操控 DOM。

优点：

- demo 很惊艳。
- 探索空间大。

缺点：

- 安全差。
- 设计系统容易崩坏。

- 可测试性弱。
- 权限边界不清。
- 难产品化。

适合：

- 原型探索。
- 沙盒实验。
- 代码生成工具，而不是运行时主路径。

成熟路线应是：

Controlled -> Declarative -> Protocolized Agentic UI -> Limited Open-ended Sandbox

4. 总体架构：双运行时、三条链路、四个闸门

4.1 双运行时

系统由两个运行时组成。



不要把它们混成一个“万能 AI 控制器”。它们职责不同。

4.2 三条链路



4.3 四个闸门



5. 全局数据流

用户表达目标
自然语言 / 语音 / 默认 UI /
自动化

图例

橙色虚线边框 = 需要或可以
调用大模型 API
普通节点 = 确定性工程模块
所有执行都必须经过契约、
策略、日志

一、输入标准化

意图输入适配器
统一来源、用户、目标、载
荷

上下文构建器
生成规划上下文与策略上下
文

二、契约注册中心

注册表与协议
Capability Graph /
Component Cards
DataSource Registry /
Design Tokens
Agent-UI Protocol / Event
Bus

后续 Planner、Validator、
Renderer、Action Router
都只能读取这里暴露的契约

图 1：修正版中文 Mermaid 全局数据流图。新版补充契约注册中心、动作路由器、数据代理引擎、流式解析与增量 Patch、UX 风险闭环和默认回退 UI；橙色虚线边框表示需要或可以调用大模型 API 的模块。
已渲染 PDF：mature-intent-native-generative-ui-build-guide1-mermaid.pdf

6. 模块总览

模块	职责	不能做
Intent Input Adapter	把自然语言、语音、按钮、API 统一成 IntentInput	不做业务执行
Context Builder	构造 planner context 和 policy context	不泄露 secret 给模型
LLM Gateway	兼容 OpenAI、DeepSeek、本地模型	不直接改状态
Semantic Planner	生成 IntentPlan	不拥有执行权
Capability Graph	注册系统可执行能力	不允许隐式能力
Plan Validator	校验 plan	不猜测修复危险 plan
Policy Engine	裁决权限、风险、确认	不听从模型风险结论
Transactional Executor	执行已授权能力	不解析自然语言
State Kernel	唯一事实源	不被 UI 直接绕过
UI DSL Generator	生成声明式 UI	不输出任意代码
DSL Validator	校验组件、props、action、dataSource	不让未知字段影响执行

模块	职责	不能做
Component Factory	DSL 节点转真实组件	不执行组件内任意脚本
Layout Solver	求解布局和响应式约束	不允许遮挡系统确认层
UX Validator	检查体验风险	不只看 schema
Dynamic Canvas	渲染动态 UI	不直接访问数据库
Execution Journal	记录全过程	不记录 secret

7. 推荐目录结构



planning/

SemanticPlanner.ts

RulePlanner.ts

ModelPlanner.ts

plannerPrompt.ts

planner.types.ts

validation/

PlanValidator.ts

plan.schema.ts

plan.types.ts

policy/

PolicyEngine.ts

riskModel.ts

confirmationPolicy.ts

permissionModel.ts

execution/

TransactionalExecutor.ts

compensation.ts

executor.types.ts

journal/

ExecutionJournal.ts

JournalRedaction.ts

generative-ui-runtime/

dsl/

uiDsl.schema.ts

uiDsl.types.ts

UIDslGenerator.ts

DSLValidator.ts

components/

ComponentRegistry.ts

componentCards.ts

ComponentFactory.ts

layout/

LayoutSolver.ts

layout.schema.ts

responsiveRules.ts

ux/

UXValidator.ts

overlayRules.ts

accessibilityRules.ts

visualRegression.ts

canvas/

DynamicCanvas.tsx

VirtualUITree.ts

streamingPatch.ts

capabilities/

CapabilityGraph.ts

capability.types.ts

domains/

ui.capabilities.ts

data.capabilities.ts

file.capabilities.ts

task.capabilities.ts

integration.capabilities.ts

state/

StateKernel.ts

StateBus.ts

stateDiff.ts

data/

DataSourceRegistry.ts

DataEngine.ts

dataSource.types.ts

protocols/



依赖方向：



8. 核心协议 1: IntentInput

所有入口都要先变成 IntentInput。

```
export type IntentInput = {  
  id: string;  
  sources:  
    | "natural_language"  
    | "voice"  
    | "command_palette"  
    | "gui"  
    | "shortcut"  
    | "api"  
    | "automation"  
  text?: string;  
  commandId?: string;  
  targetRef?: EntityRef;  
  payload?: Record<string, unknown>;  
}
```

```
actor: ActorRef;
```

```
sessionId: string;
```

```
locale: string;
```

```
timestamp: string;
```

原则：

- GUI 按钮也进入 Intent Runtime。
- 不要让传统 UI 绕过策略系统。
- 自动化触发也必须有 actor 和权限上下文。

9. 核心协议 2：Context

Context 分两份。

```
PlannerContext:
```

```
给模型看，脱敏、摘要化、只包含可见能力和组件。
```

```
PolicyContext:
```

```
给策略引擎看，完整、确定、包含真实权限和资源状态。
```

```
export type PlannerContext = {
```

```
visibleCapabilities: CapabilityCard[];
```

```
visibleComponents: ComponentCard[];
```

```
visibleDataSources: DataSourceCard[];
```

```
designTokens: DesignTokenCatalog;
```

```
focus:
```

```
  surfaceId?: string;
```

```
  entityRefs: EntityRef[];
```

```
  selectionSummary?: string;
```

```
};
```

```
stateSummary: {
```

```
  activeMode?: string;
```

```
  activeWorkspace?: string;
```

```
  unsavedChanges: boolean;
```

```
  runningTaskCount: number;
```

```
};
```

```
constraints: {
```

```
  privacyMode: "local" | "cloud_allowed" | "cloud_required";
```

```
  maxCostTier: "free" | "low" | "medium" | "high";
```



```
}:
```

```
}
```

关键：

- 不要把 secret、token、完整隐私数据给模型。
- 给模型的是“可规划视图”，不是完整数据库。

10. 核心协议 3：IntentPlan

IntentPlan 是自然语言到可执行世界的第一份结构化产物。

```
export type IntentPlan = {  
  id: string;  
  inputId: string;  
  goal: string;  
  confidence: "low" | "medium" | "high";  
  assumptions: string[];  
  steps: IntentPlanStep[];  
  clarification?: string;  
  question: string;  
  choices?: Array<{ id: string; label: string }>  
}
```

```
    }  
  
    unsupportedReason?: string;  
  }  
}  
  
export type IntentPlanStep =  
  
  {  
    id: string;  
  
    capabilityId: string;  
  
    input: Record<string, unknown>;  
  
    reason: string;  
  
    dependsOn?: string[];  
  }  
}
```

Plan Validator 检查：

- step 数量上限。
- capability 是否注册。
- input 是否符合 schema。
- dependsOn 是否成环。
- 是否引用不可见能力。
- 是否存在禁止组合。
- 是否包含未知危险字段。

11. 核心协议 4: Capability

Capability 是软件“用户有权控制的一切”的正式契约。

```
export type CapabilityDefinition = {  
  id: string;  
  
  domain: "ui" | "data" | "file" | "task" | "integration" | "settings" | "automation";  
  
  title: string;  
  
  description: string;  
  
  inputSchema: JsonSchema;  
  
  outputSchema: JsonSchema;  
  
  requiredContext: string[];  
  
  requiredPermissions: string[];  
  
  risk: RiskProfile;  
  
  effects: EffectProfile;  
  
  reversible: boolean;  
  
  execute: CapabilityExecutor;  
}
```

示例：



没有注册的能力，模型不能调用。

12. 核心协议 5：UI DSL

UI DSL 是模型和渲染器之间的界面协议。

错误路线：

```
<div style="position:absolute; left:317px" onclick="deleteAll()">...</div>
```

成熟路线：

```
"version": 1
```

```
"intentPlanId": "plan_001"
```

```
"layout": {
```

```
  "strategy": "split",
```

```
  "regions": [
```

```
    { "id": "main", "role": "primary", "preferredSize": 0.62 },
```

```
    { "id": "inspector", "role": "inspector", "preferredSize": 0.38 }
```

```
  ],
```

```
"root": {
```

```
  "id": "incident-panel",
```

```
  "component": "Panel",
```

```
  "props": {
```

```
    "title": "异常日志",
```

```
    "tone": "danger",
```

```
    "density": "comfortable"
```

```
  }
```

```
"dataSource": {
```

```
"id": "server_errorlogs"
```

```
"params": { "range": "latest" }
```

```
}
```

```
"children": [
```

```
,
```

```
"id": "logs",
```

```
"component": "LogViewer",
```

```
"props": { "maxLines": 200 }
```

```
,
```

```
,
```

```
"id": "restart"
```

```
"component": "Button"
```

```
"props": { "label": "重启服务", "tone": "danger" }
```

```
"actions": {
```

```
"click": {
```

```
"capabilityId": "server.restart"
```

```
"input": { "target": "current" }
```



注意：

- `Button` 不执行代码。
- `actions.click` 只是 `ActionRef`。
- 点击后必须走 `Action Router` 和 `Policy Engine`。

13. 核心协议 6：Component Card

模型不需要知道你的 `React/Vue` 组件源码，只需要组件卡片。

```
export type ComponentCard = {  
  component: string;  
  description: string;  
  visualRole:  
    | "container"  
    | "display"
```

```
    | "input"  
  
    | "navigation"  
  
    | "feedback"  
  
    | "chart"  
  
    | "media"  
  
    allowedProps: JsonSchema;  
  
    allowedChildren?: string[];  
  
    allowedActions?: string[];  
  
    designGuidance: string[];  
  
}
```

示例：

```
{  
  
  "component": "Panel",  
  
  "description": "A bounded surface for related controls or information.",  
  
  "visualRole": "container",  
  
  "allowedChildren": ["Text", "Button", "Table", "Chart", "LogViewer"],  
  
  "allowedProps": {  
    | "input"  
    | "navigation"  
    | "feedback"  
    | "chart"  
    | "media"  
    allowedProps: JsonSchema;  
    allowedChildren?: string[];  
    allowedActions?: string[];  
    designGuidance: string[];  
  }  
}
```



```
"tone": ["neutral", "info", "success", "warning", "danger"]
```

```
"density": ["compact", "comfortable"]
```

```
],
```

```
"designGuidance":
```

```
"Use Panel for grouped operational content."
```

```
"Do not nest Panel inside Panel."
```

```
"Use tone=danger only for real risk or error contexts."
```

14. 核心协议 7：Design Tokens

高品味不是让模型自由调像素，而是给模型一个高质量设计语言。

允许模型选择：

```
"tone": "warning"
```

```
"density": "compact"
```

```
"emphasis": "medium"
```

```
"motion": "reduced"
```

禁止模型输出：



Design Tokens 至少包括：

- semantic colors。
- spacing scale。
- typography scale。
- radius scale。
- elevation scale。
- density。
- motion preference。
- accessibility modes。

15. 核心协议 8：DataSource

动态 UI 不能直接访问数据库。

```
export type DataSourceDefinition = {  
  
  id: string;  
  
  title: string;  
  
  description: string;  
  
  paramsSchema: JsonSchema;  
  
  resultSchema: JsonSchema;  
  
  requiredPermissions: string[];  
  
  privacy: "public" | "internal" | "private" | "sensitive";  
  
  read: DataSourceReader;  
  
}
```

UI DSL 只引用:

```
{  
  
  "dataSource": {  
  
    "id": "monthly_stats"  
  
    "params": { "month": "current" }  
  
  }  
  
}
```

Data Engine 负责：

- 权限。
- 脱敏。
- 分页。
- 缓存。
- 错误状态。
- 流式更新。

16. Policy Engine

Policy Engine 是系统的主权层。它不信任模型。

输入：

- actor。
- PolicyContext。
- normalized IntentPlan。
- CapabilityDefinition。
- DataSourceDefinition。
- organization policy。
- runtime mode。

输出：

```
export type PolicyDecision =  
| { type: "allow"; planId: string }
```

```
| { type: "clarify"; question: string; choices?: ClarificationChoice[] }
| { type: "confirm"; request: ConfirmationRequest }
| { type: "deny"; reason: string; recovery?: string }
```

风险矩阵：

风险	示例	默认处理
none	打开面板、聚焦视图	allow
low	可撤销布局、局部过滤、只读查询	allow
medium	长任务、外部读取、偏好变更	confirm 或 allow
high	删除、覆盖、上传、重启、付费	confirm
critical	权限变更、不可恢复删除、跨组织迁移	deny 或强确认

17. UX Validator

来自 UXBench 论文的重要启发是：UI 正确渲染不代表 UX 合格。

UX Validator 应检查：

- 文本是否被浮层遮挡。
- 可点击区域是否被遮挡。
- 弹窗是否缺少关闭入口。
- 是否出现多个模态弹窗堆叠。

- 原生关闭按钮是否不可点击。
- 宣传文案和实际内容是否不一致。
- 服务描述和页面功能是否不一致。
- 确认按钮是否被动态 UI 遮挡。
- reset/default UI 是否仍可达。

实现可分三层：

规则层：DOM/Layout 静态检查

视觉层：截图 + bounding boxes

多模态层：MLLM/UX model 做体验推理

MVP 先做规则层和视觉回归，不必一开始训练 UX 模型。

18. Streaming UI

生成式 UI 最大体验问题是延迟。

推荐流程：

0ms：用户输入

100ms：显示“正在理解”

300ms：显示 plan preview

500ms：显示布局骨架

800ms+：组件逐步挂载

数据完成后：局部组件更新

高风险动作：始终等用户确认

技术：

- structured object streaming。
- partial JSON parser。
- incremental UI tree patch。
- skeleton components。
- component-level loading。
- optimistic layout with pessimistic action execution。

原则：

- UI 可以乐观渲染。
- 动作不能乐观执行高风险操作。

19. Human-in-the-loop

确认界面不应只是“确定/取消”。

它必须解释：

- 我理解你要做什么。
- 我将执行哪些步骤。
- 会影响哪些资源。
- 数据是否离开本地。
- 是否产生费用。

- 是否可撤销。
- 失败后如何恢复。

确认请求：

```
export type ConfirmationRequest =
```

```
  planId: string;
```

```
  summary: string;
```

```
  steps: Array<|
```

```
    title: string;
```

```
    resourceSummary: string;
```

```
    riskSummary: string;
```

```
  |>;
```

```
  dataMovement: "none" | "local_only" | "external";
```

```
  reversible: boolean;
```

```
  confirmLabel: string;
```

```
  cancelLabel: string;
```

```
};
```


20. 审计系统

Execution Journal 记录:

```
export type ExecutionJournalEntry = {  
  id: string;  
  
  inputSummary: string;  
  
  contextHash: string;  
  
  intentPlan: IntentPlan;  
  
  uiDsl?: UIDslDocument;  
  
  validation: .  
  
  plan: "passed" | "failed";  
  
  dsl: "passed" | "failed";  
  
  ux: "passed" | "failed" | "warning";  
  
  !:  
  
  policyDecision: PolicyDecision;  
  
  events: RuntimeEvent[];  
  
  stateDiffSummary?: string;  
  
  startedAt: string;
```

```
endedAt?: string;
```

隐私要求：

- 不记录 secret。
- 不记录完整敏感数据。
- 使用 context hash。
- 高隐私模式下 journal 只保存在本地。

21. 手把手构建：第 0 步，先画边界

不要先接模型。

先列四张清单：

1. 用户能控制什么？

2. 系统能显示什么组件？

3. 系统有哪些数据源？

4. 哪些动作危险？

输出：

- capabilities.yaml
- components.yaml
- dataSources.yaml
- riskRules.yaml

验收：

- 每个用户可控动作都有 capabilityId。
- 每个动态组件都有 component card。
- 每个数据访问都有 dataSourceId。
- 每个高危动作都有风险定义。

22. 第 1 步，建立固定组件库

MVP 组件不要超过 10 个：



每个组件必须定义：

- props schema。
- allowed children。
- allowed actions。
- loading state。
- error state。
- empty state。
- accessibility behavior。

验收：

- 组件不用 AI 也能稳定渲染。
- 所有 props 都能被 schema 校验。

23. 第 2 步，定义 UI DSL

先支持最小 DSL：





限制：

- 最大节点数：例如 80。
- 最大深度：例如 6。
- 禁止任意 style。
- 禁止任意 script。
- 禁止未知 component。
- 禁止未知 action。

验收：

- 手写一份 DSL 可以渲染。
- 错误 DSL 会被拒绝。

24. 第 3 步，实现 Dynamic Canvas

Dynamic Canvas 接收 UI DSL，渲染组件树。

流程：

DSL -> validate -> normalize -> virtual tree -> layout -> render

Canvas 必须支持：

- loading。
- error boundary。
- reset to default。
- component fallback。
- action interception。
- responsive layout。

验收：

- 组件树变化时界面更新。
- 任何组件错误不会炸掉整个应用。
- reset 按钮始终可见。

25. 第 4 步，实现 Action Router

所有动态 UI 事件进入 Action Router。

```
click -> ActionRef -> Capability -> Policy -> Executor
```

ActionRef 示例：

```
"capabilityId": "file.export",
```

```
"input": { "format": "pdf", "scope": "current" }
```

验收：

- 动态按钮不能直接调用 API。
- 高危 action 会出现确认界面。
- 未注册 action 被拒绝。

26. 第 5 步，实现规则 Planner

在 LLM 之前，先用规则 planner 打通链路。

支持命令：

打开设置

切换到专注模式

把当前数据画成柱状图

导出当前报告

验收：

- 不接模型也能生成 IntentPlan 和 UI DSL。
- 模糊命令会澄清。

27. 第 6 步，接入 LLM Gateway

LLM Gateway 统一模型接口。

```
export type LLMGateway =
```

```
createIntentPlan(input: PlannerRequest): Promise<IntentPlan>
```

```
createUITDs!(input: UITDs!Request): Promise<UITDs!Document>
```

推荐策略：

- 先用 structured output。
- 输出失败时 retry 一次。
- 仍失败则 fallback 到规则 planner。
- 只传 capability cards 和 component cards。

Prompt 必须包含：

- 用户输入。
- 当前上下文摘要。
- 可用能力。
- 可用组件。
- design tokens。
- 输出 schema。
- 禁止事项。

28. 第 7 步，实现 Policy Engine

先写确定性规则。

规则示例：

```
file.delete -> high risk -> confirm
```

```
server.restart -> high risk -> confirm
```



```
data.query(public) -> low risk -> allow
```

```
ui.layout.applyPreset -> low risk -> allow
```

```
integration.sync(external) -> high risk -> confirm
```

```
unknown capability -> deny
```

验收：

- 模型说 low risk 不算数。
- Policy Engine 自己计算风险。

29. 第 8 步，实现 UX Validator

MVP 规则：

- 禁止动态 UI 遮挡系统确认层。
- 禁止多个 modal 同时 active。
- modal 必须有关闭路径。
- reset 按钮始终可见。
- 关键按钮最小点击尺寸。
- 文本不溢出容器。

生产级：

- screenshot diff。
- accessibility audit。
- bounding box overlap detection。
- 多模态 UI reasoning。

30. 第 9 步，实现 Streaming UI

先不要追求复杂流式 JSON。

推荐三阶段：



验收：

- 用户 500ms 内看到反馈。
- 长任务有进度。
- 数据组件可独立 loading。

31. 第 10 步，建立测试矩阵

测试不是只测模型。

测试类型	测什么
Golden Intent	自然语言是否转成正确 plan
Plan Validator	错误 plan 是否被拒绝
DSL Validator	错误 UI DSL 是否被拒绝
Policy Test	越权/高危是否拦截

测试类型	测什么
Executor Test	执行、失败、回滚
UX Test	遮挡、弹窗、可达性
Visual	生成 UI 是否稳定
Regression	
Prompt	是否能绕过规则
Injection	

攻击样例：

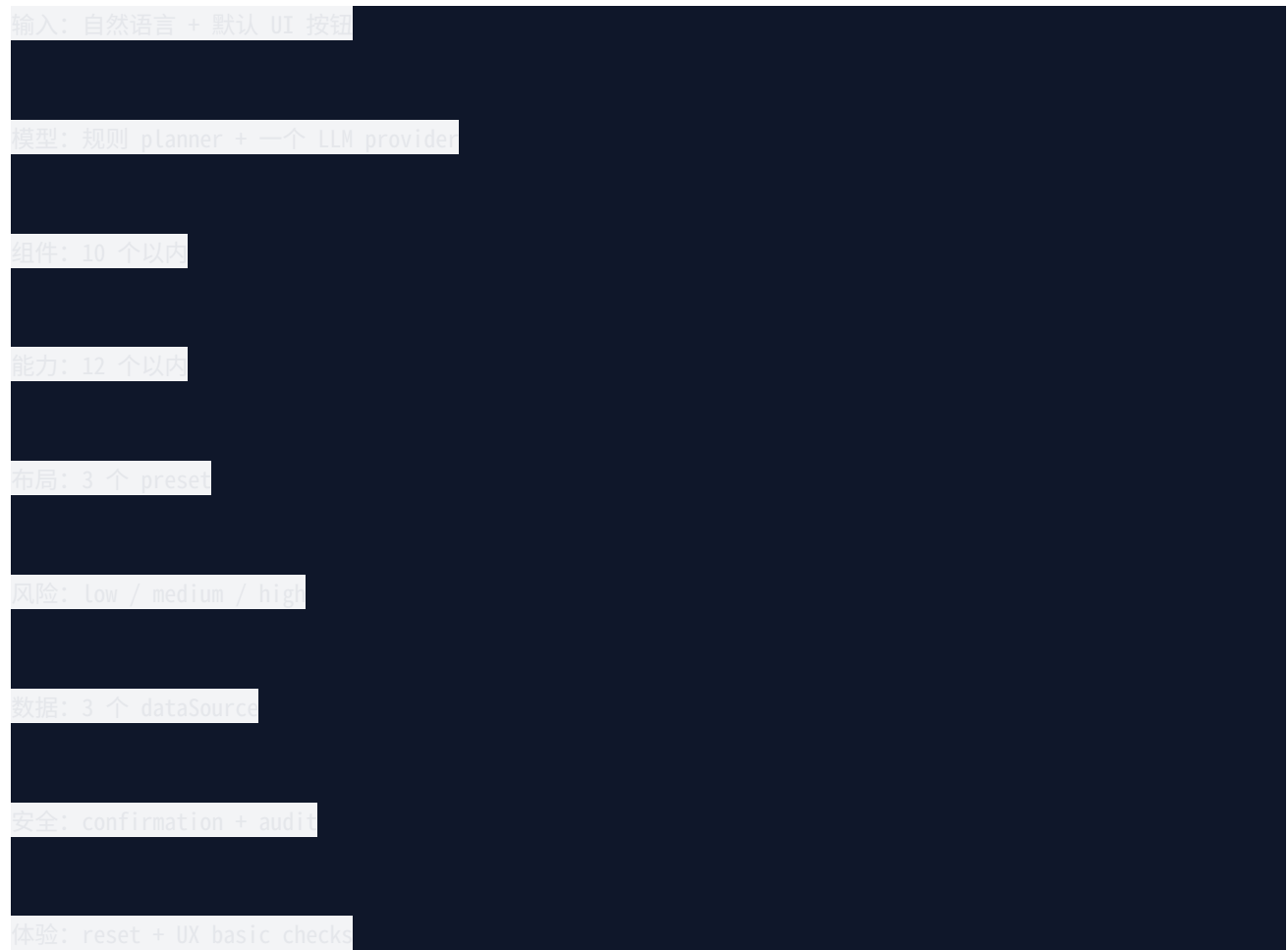


期望：

- schema 拒绝。
- policy 拒绝。
- executor 没有入口。
- journal 记录攻击输入。

32. MVP 蓝图

MVP 只做这些：



MVP 用户故事：



3. 查询允许的数据源。

4. 生成 UI DSL。

5. 右侧打开 Panel。

6. 渲染 Chart。

7. 审计记录整个过程。

33. 生产级演进路线

Phase 0: Registry First

组件、能力、数据源、风险规则。

Phase 1: DSL Runtime

手写 DSL 可渲染，错误 DSL 可拒绝。

Phase 2: Intent Runtime

规则 planner + validator + policy + executor。

Phase 3: LLM Planner

structured output + fallback + retry。

Phase 4: Controlled GenUI

preset layout + tokenized theme + registered components。

Phase 5: Declarative GenUI

schema-driven component tree + layout solver。

Phase 6: UX Governance

UX validator + visual regression + accessibility audit。

Phase 7: Agent-UI Protocol

runtime events、tool invocation、state bridge。

Phase 8: Plugin Ecosystem

插件注册组件、能力、workflow，但不能绕过 policy。

34. 关键工程取舍

34.1 模型选型

不要让模型选择架构。模型只是 provider。

系统应支持：

- OpenAI。
- DeepSeek。
- 本地模型。
- 规则 planner。
- 领域 planner。

核心是输出协议稳定。

34.2 Web 还是桌面

Web 优先适合验证：

- 动态布局成熟。
- 组件生态丰富。
- streaming UI 容易做。

桌面适合产品化：

- 本地文件。
- 系统 API。
- 强沙箱。
- 离线能力。

推荐：



34.3 什么时候允许开放式生成

只在沙盒里允许：

- 原型生成。
- 临时预览。
- 用户确认后保存为组件。
- 不直接进入生产执行链路。

35. 最终架构口号



这就是目前最成熟的通用范式。